An Uncertainty-Aware, Explainable Hybrid Metaheuristic Framework for Multi-Objective Task Scheduling in Heterogeneous Fog-Cloud Systems

A Full-Stack Distributed Platform Extending Bio-Inspired Optimization and Conformal Inference

Shri Srivastava (2023ebcs593) | **Ichha Dwivedi** (2023ebcs125) | **Aditi Singh** (2023ebcs498)
Project Supervisor: Swapnil Saurav | **Academic Affiliation:** BITS Pilani Online (BCS ZC241T)
Repository: https://github.com/shri33/ML-Task-Scheduler

ABSTRACT — *The rapid proliferation of Industrial Internet of Things (IIoT) devices has introduced severe computational bottlenecks at the network edge. Allocating heterogeneous computational tasks across multi-tiered Fog-Cloud architectures under strict latency, energy, and deadline constraints is an NP-hard optimization challenge. Traditional scheduling heuristics rely on idealized, static execution assumptions, leading to high Service Level Agreement (SLA) violation rates during dynamic edge workload fluctuations. Furthermore, black-box machine learning approaches fail to quantify prediction uncertainty, exposing edge nodes to catastrophic scheduling failures. In this paper, we propose a comprehensive, uncertainty-aware, explainable hybrid task scheduling framework. Our architecture integrates: (1) an* ***Improved Particle Swarm Optimization and Improved Ant Colony Optimization (IPSO-IACO) Hybrid Heuristic*** *that dynamically balances global exploration and local path exploitation, (2) an* ***Ensemble Machine Learning Inference Engine*** *predicting task runtimes with sub-15ms latency (R^2 = 0.9483, MAE = 0.214s), (3)* ***Split Conformal Prediction*** *providing distribution-free 90% confidence bounds on execution duration, and (4)* ***SHAP (SHapley Additive exPlanations)*** *delivering real-time decision interpretability. We implement the complete system as an open-source, containerized microservices platform comprising a React 18 frontend, a Node.js API gateway with asynchronous BullMQ Redis worker queues, a Python/Flask MLOps engine, and a 30-entity PostgreSQL schema. Empirical evaluations replicating the foundational benchmark of Wang & Li (2019) across workloads of 50 to 300 tasks demonstrate that our proposed Hybrid Heuristic reduces completion delay by 18.4–38.2% and terminal energy consumption by 14.1–29.6% compared to classical FCFS, Round-Robin, and Min-Min baselines, while maintaining an empirical conformal coverage rate of 91.2% at $\alpha = 0.10$.*

Keywords: Fog Computing, Task Scheduling, Hybrid Metaheuristics, Particle Swarm Optimization, Ant Colony Optimization, Machine Learning, Conformal Prediction, Explainable AI, IIoT.

1. Introduction & Theoretical Motivation

In time-critical cyber-physical systems—such as automated manufacturing lines, smart grids, and real-time medical monitoring—offloading computational tasks entirely to centralized cloud data centers incurs prohibitive wide-area network (WAN) latency (50–150ms) and backhaul bandwidth saturation. Conversely, resource-constrained edge devices lack the CPU and battery power for local execution. Fog computing introduces virtualized compute nodes in close physical proximity to edge devices (5–20ms latency), yet scheduling N heterogeneous tasks onto M distributed heterogeneous nodes represents an NP-hard combinatorial problem with M^N solution permutations.

Existing scheduling systems exhibit three fundamental limitations: (1) *Static Heuristics* (FCFS, Min-Min) assume idealized constant task durations, ignoring CPU contention and OS overheads; (2) *Standalone Metaheuristics* (PSO, ACO) suffer from either premature convergence in local optima or slow initial exploration; (3) *Simulation Isolation* restricts prior literature to theoretical scripts (e.g., CloudSim) without delivering deployable, reactive microservice architectures. Our work resolves these limitations through an integrated full-stack platform.

2. Fog Computing Mathematical Optimization Model

We implement the exact 3-layer mathematical offloading formulation from **Wang & Li (2019)** (Sensors 19(5), 1023):

- **Task Execution Delay:** $T_{Eij} = (D_i \times 10^6 \times 8 \times \theta_i) / C_j$
- **Network Transmission Delay:** $T_{Tij} = L_j + (D_i / B_j)$ *[$T_{Tij} = 0$ for local execution]*
- **Total Task Completion Delay:** $T_{Dij} = T_{Tij} + T_{Eij} + S_i$
- **Terminal Device Energy Consumption:** $E_{ij} = (T_{Tij} \times P_T) + (T_{Eij} \times P_{idle})$
- **Composite Multi-Objective Cost Function:** $J(X) = \Sigma\ [\ w_{delay}\ T_{Dij} + w_{energy}\ E_{ij} + 10 \cdot Cost_{egress} + \Phi_{ij}\ ]$
- **Global Optimization Fitness:** $Fitness(X) = 1 / (J(X) + \varepsilon)$

3. Two-Stage Bio-Inspired Hybrid Metaheuristic (HH)

To achieve rapid global convergence without stagnation, our Hybrid Heuristic executes in two coordinated stages:
Stage 1 (IPSO Global Exploration): Particles explore the global allocation space for I_1 iterations using a non-linear adaptive inertia weight: $w(t) = w_{min} + (w_{max} - w_{min}) \times \exp(-20 \times (t / T_{max})^2)$ with $w_{max} = 0.9$ and $w_{min} = 0.4$.
Stage 2 (IACO Local Refinement): The best global solution from IPSO seeds the initial pheromone distribution τ_0 for IACO, preventing blind ant wandering. IACO refines allocations for I_2 iterations using Max-Min ant system bounds $[\tau_{min}, \tau_{max}]$ and multi-objective heuristic desirability $\eta_{ij} = 1 / (T_{Dij} + \omega\ E_{ij})$.

4. Uncertainty-Bounded & Explainable ML Pipeline

The Python ML microservice employs an ensemble of **XGBoost** and **Random Forest Regressors** trained on empirical cloud task traces and physical execution characteristics. To ensure high-stakes edge reliability, point predictions are wrapped with **Split Conformal Prediction**: computing non-conformity residuals $R_k = |y_k - f(x_k)|$ on a held-out calibration set yields a distribution-free $(1-\alpha)$ quantile bound $\hat{q}$. For $\alpha = 0.10$, the interval $[\hat{y} - \hat{q}, \hat{y} + \hat{q}]$ statistically guarantees 90% coverage of ground-truth execution times. Simultaneously, **SHAP TreeExplainer** computes additive feature attributions exposing exactly how task size, priority, and node load drive predictions.

Table 1: Machine Learning Regression & Inference Benchmarks

Model	R^2 Score	MAE (s)	RMSE (s)	Latency (ms)
Linear Regression	0.6120	0.842	1.120	0.8 ms
Decision Tree	0.8814	0.354	0.541	1.2 ms
Gradient Boosting	0.9245	0.268	0.412	3.8 ms
XGBoost Regressor	**0.9483**	**0.214**	**0.328**	**4.5 ms**
Random Forest (Default)	0.9412	0.228	0.345	5.2 ms

5. Empirical Validation & Wang & Li (2019) Benchmark Reproduction

We evaluated 10 scheduling algorithms across identical 10-node heterogeneous fog topologies under workloads of 50 to 300 tasks and deadline tolerances from 10s to 100s. Results confirm that the proposed Hybrid Heuristic achieves substantial improvements across all objectives.

Table 2: Comparative Makespan (Seconds) and Energy (Joules) across Workload Volume N

Algorithm	N=50 (Time/J)	N=100 (Time/J)	N=200 (Time/J)	N=300 (Time/J)	Makespan Gain
FCFS	12.4s / 42.1J	28.6s / 96.4J	69.4s / 228.6J	118.5s / 389.2J	Baseline
Round-Robin	11.8s / 39.8J	26.2s / 88.7J	64.8s / 214.1J	111.2s / 367.8J	+6.2%

Min-Min	9.2s / 31.2J	21.4s / 72.4J	53.2s / 178.5J	92.4s / 310.4J	+22.0%
Standalone IPSO	8.4s / 28.5J	18.9s / 64.1J	46.1s / 154.8J	79.8s / 268.1J	+32.6%
Standalone IACO	8.1s / 27.8J	18.2s / 62.5J	44.3s / 149.2J	76.5s / 258.4J	+35.4%
Proposed HH	**7.1s / 24.1J**	**15.4s / 53.8J**	**37.2s / 127.4J**	**64.2s / 219.5J**	**+45.8% (vs FCFS)**

6. Full-Stack Microservices Architecture & Engineering

The complete system is containerized via Docker Compose across 7 services:
• **Client Layer (Port 3000):** React 18 SPA with Vite, Tailwind CSS, Zustand client stores, TanStack React Query, and Socket.IO client.
• **Application Gateway (Port 3001):** Node.js Express server with 19 modular route controllers (70+ REST endpoints), Double-Submit CSRF cookies, and 6-tier Redis rate limiters.
• **Asynchronous Worker Tier:** BullMQ Redis queue workers (prediction, scheduling, notification, autoRetrain) with Dead Letter Queues (DLQ).
• **Data Persistence:** PostgreSQL 15 with 30 normalized Prisma models and multi-column indexes for fast priority-queue retrieval.

7. Conclusion

This research presented an uncertainty-aware, explainable hybrid task scheduling framework for heterogeneous Fog-Cloud environments. By uniting non-linear IPSO-IACO metaheuristics with sub-15ms XGBoost duration predictions, Split Conformal Prediction uncertainty intervals (91.2% empirical coverage), and real-time SHAP interpretability, the system achieves a 38.2% reduction in completion time and 43.6% reduction in terminal energy consumption over standard heuristics. The open-source codebase provides a robust, reproducible foundation for distributed edge computing research.

Key References

[1] J. Wang and D. Li, 'Task scheduling based on a hybrid heuristic algorithm for smart production line with fog computing,' *Sensors*, vol. 19, no. 5, p. 1023, 2019.

[2] F. Bonomi et al., 'Fog computing and its role in the Internet of Things,' in *Proc. ACM MCC Workshop*, 2012, pp. 13–16.

[3] A. N. Angelopoulos and S. Bates, 'A gentle introduction to conformal prediction,' *arXiv:2107.07511*, 2021.

[4] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[5] T. Chen and C. Guestrin, 'XGBoost: A scalable tree boosting system,' in *Proc. ACM KDD*, 2016, pp. 785–794.